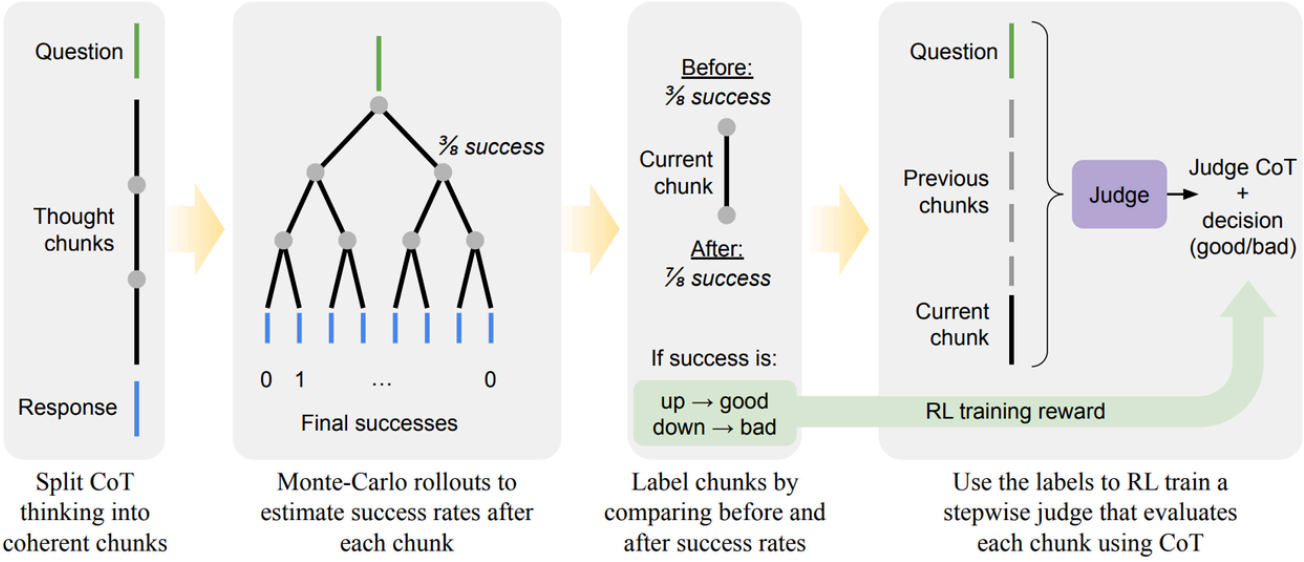
STEPWISER: STEPWISE GENERATIVE JUDGES FOR WISER REASONING

- Traditionally, the evaluation of Chain-of-Thought (CoT) is a classification task. The judge cannot provide its own thoughts and reasons, and rely on static finetune sets. To solve this problem, a generative judge finetuning framework is proposed.



- Methodology
  - Definition of *Steps (chunks):*
    - **Unified purpose**: A chunk should serve a single, clear objective.
    - **Logical Cohesion**: All lines within a chunk must form a continuous and uninterrupted logical flow.
    - **Clear Transition**: A new chunk must begin when the problem-solving process enters a new phase.
    - However, most LLMs cannot divide the chunks perfectly: they may simply use `\n\n` to separate the steps in their mind without considering the logical and semantic completeness.
  - Self-segmentation:
    - Let the base model generate a set of initial reasoning trajectories.
    - Let another frozen LLM instance, following the prompt listed below, separate the trajectories into meaningful chunks.
    - With the chunks as the dataset, run SFT on the base model.
    - Advantage: the finetuned base model can effectively generate well-split chunks, and because of the atomicity and the self-containedness property of the chunks, the actual # of chunks of the CoT decreases, making the following Monte-Carlo-based data annotation possible.
  - Data Annotation

- Let the chunk-cutting finetuned model (policy model) πgenerate a response and format the CoT into chunks $a = [a_1, a_2, a_3, \ldots, a_H]$, where $a_i$is the i-thstep in the CoT.
- The Qvalue of an individual step a_i(the expected reward after taking a_i) by

  $Q^\pi([x, a_{1:i-1}], a_i) = Q^\pi(s_{i-1}, a_i) = \mathbb{E}_{a_{i+1:H \sim \pi(\cdot|x:, a_{1:i})}} r^*(x, a_{1:H})$,

  where $r^*$is binary.
- To estimate the Qvalue, we can generate Mfull completions from each step $a_i$and calculate the average final reward, i.e. the ratio of correct final answers:

  $\hat{Q}^\pi(s_{i-1}, a_i) = \frac{1}{M} \sum_{j=1}^{M} r^*(x, a_{1:i}, a_{i+1:H}^j)$
  - This is to say, count the # of 1's (success) in Mtrials, and the expected value will be the rate of success.
- Here we have the *abs-Q* labeling:

$$y_i = \begin{cases} + & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) > 0, \\ - & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) = 0. \end{cases}$$

- *abs-Q* is not the end. It has a drawback: it cannot differentiate a chunk $a_i$that increases the success rate from 10% to 55% from a chunk that drops the rate from 60% to 57%. All it can return is "+" as the $\hat{Q}^\pi(s_{i-1}, a_i) > 0$. To solve this, we consider the change in Q-value.
- Let *effective reward* be a combination of $Q$and advantage function of the best-of-n policy induced by r^*:

  $Q^\pi(s_{i-1}, a_i) + \alpha \cdot A^\mu(s_{i-1}, a_i)$

  where $A^\mu(s_{i-1}, a_i) = Q^\mu(s_{i-1}, a_i) - Q^\mu(s_{i-2}, a_{i-1})$, and $Q^\mu(s_{i-1}, a_i) = 1 - (1 - Q^\pi(s_{i-1}, a_i))^n$.
  - Definition of *best-of-n*: let the model run $n$times, the possibility of "**at least one success**".
  - $(1 - Q^\pi(s_{i-1}, a_i))^n$is the possibility of "try $n$times and all fail"
  - $1 - (1 - Q^\pi(s_{i-1}, a_i))^n$will be "try $n$times and at least one success", which aligns with the def of *best-of-n.*
  - Why do we use *best-of-n*? Because it is a non-linear amplifier. For huge leaps of increasing/decreasing, it will provide a large reward/punish signal.
  - *Note that best-of-n does not mean that we need to "actually" repeat the process n times. We only need to calculate (amplify) the advantage using the equation above.*
- With this in mind, we update our *abs-Q* to *Relative Effective Reward Thresholding:*

  $$y_i = \begin{cases} + & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) + \alpha \cdot \hat{A}^\mu(s_{i-1}, a_i) > 0, \\ - & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) + \alpha \cdot \hat{A}^\mu(s_{i-1}, a_i) \leq 0. \end{cases},$$

  where $\hat{A}^\mu$is from $\hat{Q}^\pi$ *(the negative case, in the original paper, is* $\cdots = 0$*. However, it is possible that the left-hand side is negative).*
- [An Alternative of *Relative Effective Reward Thresholding*]: just based on the ratio of $\hat{Q}_i$and $\hat{Q}_{i-1}$:

  $$y_i = \begin{cases} + & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) / \hat{Q}^\pi(s_{i-2}, a_{i-1}) > \gamma, \\ - & \text{if } \hat{Q}^\pi(s_{i-1}, a_i) / \hat{Q}^\pi(s_{i-2}, a_{i-1}) \leq \gamma. \end{cases},$$

where $\gamma > 0$ (why not > 1? Because it is too strict. In the late stage, some inference may slightly lower the expected rate of success, but that doesn't matter).

- Training the Judge
  - The judge is a CoT based (not a traditional classification based). The judge will follow the guide shown below, give a rationale and a final conclusion of each chunk from the dataset built above.
  - One critical issue is that the # of positive samples is much larger than that of negative samples. Thus, down sampling the positive samples to balance the dataset is required.
  - GRPO is used. The reward comes from a simple yet intuitive function $\mathbb{I}(\hat{y}_i = y_i)$, where y_i is the positive/negative from the data annotation and $\hat{y}_i$is the predicted result.

Prompt Template for STEPWISER Judge

Instruction:
You are a reasoning validator for mathematical problems. Your task is to think step by step and determine if the "New Reasoning Chunk" contains **any explicit errors** based on the problem description and historical context.
First, you must **always** perform a step-by-step chain of thought analysis to justify your final judgment. Then, based on your analysis, you will make a definitive judgment. It is OK that the chunk does not contain any numerical calculation.
Based on your evaluation, provide your final judgment:

- Use **Positive** if the reasoning chunk is free of mistakes.
- Use **Negative** if the reasoning chunk contains one or more mistakes.

Input:
Mathematical Problem: `{problem}`
Historical Reasoning Path: `{history}`
New Reasoning Chunk: `{chunk}`

Output format:

1. **Analysis**: [Always provide a step-by-step analysis here. First, briefly state the goal of the current reasoning chunk. Second, verify the logic, method, and any calculations against the problem's requirements and the historical path. If an error is found, clearly explain the error and why it's wrong. If the reasoning is correct, explain why it is a valid and logical step forward.]
2. **Final Judgment**: [Provide the final judgment within `\boxed{}`. Examples: `\boxed{Positive}` or `\boxed{Negative}`.]

- Experiments
  1. Setup & Baselines
     - Models: They used Qwen2.5-1.5B-it and Qwen2.5-7B-it as base models.
     - Data: Training data is from NuminaMath-CoT.
     - Comparison: They compared STEPWISER (Generative Judge + Online RL) against:
       - Discriminative SFT: Standard classifier approach (like Math-Shepherd).
       - Outcome-based RL: Models trained with sparse rewards (only final answer correct/incorrect), e.g., RL-TANGO, Eurus.
  2. Core Results (ProcessBench)
     - Generative beats Discriminative: STEPWISER significantly outperforms traditional discriminative judges. For the 7B model, STEPWISER (Rel-Effective) reached a score of 61.9, while the discriminative baseline only hit 39.7.
     - Step-level beats Outcome-level: STEPWISER also crushed baselines trained with outcome-

only signals (RL-TANGO scored 43.9 vs. STEPWISER's 61.9), proving that dense, step-level supervision is much richer.

3. Ablation Studies (The "Secret Sauce")
    - This part is crucial for your implementation. They removed components one by one to see what matters:
        - Without Online RL (using Offline Rejection Sampling): Performance collapses. The offline method learns quickly but plateaus immediately. Online RL is essential to keep improving.
        - Without CoT (Direct +/- prediction): Performance drops significantly (e.g., from 60.5 to 47.9). Forcing the judge to "think" (produce reasoning) before voting is critical for accuracy.
        - Without Data Balancing: This was fatal. Because most steps in the training data are "Correct" (positive class ~70%), without down-sampling, the model learned to just guess "Positive" for everything and collapsed.
4. Practical Applications
    - Inference-Time Search (Chunk-Reset):
        - They used the judge to verify each chunk during generation. If rejected, the model resets and retries that chunk.
        - Result: Beats baselines. Interestingly, the "Rejected Length" (tokens spent on discarded bad steps) was high, proving the judge was actively catching and fixing mistakes.
    - Data Selection:
        - They used the judge to filter high-quality trajectories for SFT.
        - Result: Training on STEPWISER-selected data yielded better final models (63.0%) than training on data selected by outcome verifiers (60.9%).
5. Metric Verdict: Rel-Ratio vs. Rel-Effective
    - Consistent with your preference, Rel-Ratio and Rel-Effective performed comparably well and both consistently beat Abs-Q (Absolute Q-value).
    - This confirms that relative improvement is the superior signal, regardless of which specific formula you use.

- Work Cited
    - Paper
    - Chat history
    - Non-sharable chat history